The HL7 ETL demo

A runbook for showing the HL7 v2 to FHIR pipeline end to end: messages land in Postgres integration tables, a .NET loader maps them to FHIR R4 and bulk-loads a public FHIR server, and the patient explorer reads the result back. Eight minutes, a handful of terminal commands, one browser tab. Synthetic data only.

1. A workload every health system runs

An EHR emits HL7 v2 messages (ADT^A01 admissions, ORU^R01 results); an integration engine lands them, transforms them to FHIR R4, and loads a FHIR server that the clinical tools read. Here the landing store is Postgres, the engine is the console in api/src/Hl7.Etl, the FHIR server is the public HAPI sandbox, and the reader is fhir.mvcprogrammer.com. The design is in the HL7 ETL section of api/README.md.

Stage	What happens	Where
Land	Each message is a row, unique on its control id (MSH-10), so a redelivery lands once. The loader claims batches in arrival order with `FOR UPDATE SKIP LOCKED`; the landing table is the only record of progress.	`Sql/001_landing.sql`
Parse and map	PID, PV1, OBR, and OBX become Patient, Encounter, and Observation with LOINC codes, UCUM units, and identifiers typed MR and VN. A message that cannot be mapped names its control id, field, and reason; the batch goes on without it.	`Mappers/`
Load	One transaction bundle per batch, every entry a conditional create keyed on the ETL identifier, through a resilient HttpClient. Replay the feed and the server reports everything already present.	`Services/LoadService.cs`
Read back	Every resource carries the identifier system `https://mvcprogrammer.com/fhir/etl`, so the cohort is one search away in the explorer.	the explorer

2. Before the demo (ten minutes, the same morning)

Step 1. Start Postgres, build the ETL, and read the landing table.

```
cd "/Volumes/Storage Hot/source/mvcprogrammer.com"
export PATH="/opt/homebrew/opt/postgresql@17/bin:$PATH"
brew services start postgresql@17
dotnet build api/src/Hl7.Etl -c Release
B=api/src/Hl7.Etl/bin/Release/net10.0/Hl7.Etl
$B status
```

Expected: `pending 0, claimed 0, processed 67, failed 0`. A new terminal loses the two variables; rerun the `export` and `B=` lines. The binary path goes in the variable, not a `dotnet run` command: zsh treats the latter as one command name.

Step 2. Check the cohort is still on the sandbox.

The sandbox is wiped now and then. Count the patients the ETL loaded:

```
curl -s -H "Accept: application/fhir+json" \
  "https://hapi.fhir.org/baseR4/Patient?identifier=https://mvcprogrammer.com/fhir/etl|&_summary=count"
```

Expected `"total": 25`. If it is 0, replay the feed, which recreates everything in a few seconds:

```
$B requeue && $B load
```

Step 3. Warm the explorer.

Open fhir.mvcprogrammer.com, clear the Family name field, put `https://mvcprogrammer.com/fhir/etl|` in the Identifier field (the bar at the end matters), click Search, and click one patient. The function caches a patient summary at the edge for ten minutes, so the summary opens instantly in the demo. Note which patient you opened.

Step 4. Open the tabs.

- The explorer, left on the search above.
- The repository folder api/src/HI7.Etl on GitHub.
- Pull request #223, the loader, whose description records what the live server taught.
- The JD cross-reference page, case study 5, if the demo is part of an interview.

3. The demo (about eight minutes)

Step 1. Frame it (thirty seconds).

“This is the integration-engine job every health system runs: HL7 v2 messages arrive from an EHR, land in integration tables, get transformed to FHIR R4, and are loaded into a FHIR server that the clinical tools read. I built it end to end on my stack in a day. Synthetic data only.”

Step 2. Show the landing tables (one minute).

```
$B status
psql -d hl7_etl -c "select message_type, trigger_event, count(*) from hl7.messages group by 1, 2 order by 1"
psql -d hl7_etl -Atc "select raw_message from hl7.messages where message_type = 'ORU' \
  order by message_id limit 1" | tr '\r' '\n'
```

Point at the segments: MSH, PID with the MRN, OBR, and the OBX lines with LOINC codes and UCUM units. Say that every message is a row, unique on its control id, so a redelivery lands once.

Step 3. Show that landing is idempotent (thirty seconds).

```
$B generate --patients 25
```

It prints `0 messages landed, 67 had already landed`. Say this is why the control id is the key.

Step 4. Replay the load (two minutes).

```
$B requeue
$B load
```

Walk the log: each batch of up to 25 messages becomes one transaction bundle, the server answers per entry, and the summary says `0 resources created, 268 already present`. Say that every entry is a conditional create keyed on an identifier, so the same feed can be loaded as many times as you like and nothing duplicates. That is the property a real engine needs after a crash or a redelivery.

Step 5. Read it back in the explorer (two minutes).

In the explorer tab, with the identifier search already run, pick the patient you warmed. Show the MRN, the encounter count, and the vital signs with their values. Open the Under the hood panel to show the FHIR calls the page made. Say the explorer did not change for this; the ETL stamps everything with one identifier system, so the cohort is one search away.

Step 6. Code tour on GitHub (two minutes).

Open three files and say one sentence each:

- `Sql/001_landing.sql`: the application calls only functions; the batch claim uses `FOR UPDATE SKIP LOCKED`, the same shape as a high-write integration table on SQL Server.
- `Mappers/AdmissionMapper.cs` or `ResultMapper.cs`: parse at the boundary, map to FHIR with the Firely SDK, and a message that cannot map names its control id, field, and reason and does not stop the batch.
- `Services/LoadService.cs`: one bundle per batch, resilient HTTP, the batch settled in the landing table.

Then pull request #223 and its section “What the first live run taught, now handled and tested”: the public server refused conditional references, and a failed batch was being re-claimed in the same run. Both found by running it for real, both fixed with tests the same afternoon. This is the part a technical audience cares about most.

Step 7. Close (thirty seconds).

The production version of this shape is the most recent position on the resume, on SQL Server at enterprise scale. Here the engine is Postgres, the server is a public sandbox, and the whole thing is public, tested, and on the project board as an epic with four reviewed pull requests.

4. If something goes wrong

Symptom	What to do
The sandbox is down	Both halves depend on hapi.fhir.org. Show the landing tables and the code tour, and run `$B load` anyway: the run log shows batches failing cleanly with the server's reason and keeping their claim for retry, which is itself a good story.
A summary shows zero observations	The ten-minute edge cache on a patient loaded moments earlier. Pick the patient you warmed, or any other.
`$B` is not found	A new terminal lost the variables; rerun the `export PATH` and `B=` lines from section 2.
The explorer lists more than 25	The identifier search is exact, so extra rows mean the Family name field still has a value or the bar is missing from the end of the identifier token. Clear the family name, keep the bar, and search again.
`generate` lands messages	The landing table was emptied (a new database, or a test run against it). Fine: `$B load` then creates on the sandbox what is not there and reports the rest as already present.

5. Links

What	Where
The ETL	api/src/Hl7.Etl
Its tests	api/tests/Hl7.Etl.Tests
Design notes	api/README.md, section HL7 ETL
The epic	Epic #216
The pull requests	#221 landing, #222 mapping, #223 loader, #224 docs
The explorer	fhir.mvcprogrammer.com
The sandbox	hapi.fhir.org/baseR4